Design and Analysis of Algorithms (DAA)

Complete Answers – 5th Semester BE

PART A – SHORT ANSWER QUESTIONS

---

1. Solve $T(n) = 4T(n/2) + n^3$ using Master's Theorem

Master's Theorem: For $T(n) = aT(n/b) + f(n)$,

- $a = 4, b = 2, f(n) = n^3$
- Compute: $\log_b(a) = \log_2(4) = \mathbf{2}$
- Compare $f(n) = n^3$ with $n^{(\log_b a)} = n^2$

Since $f(n) = n^3 = \Omega(n^{(2+\varepsilon)})$ for $\varepsilon = 1 > 0$ → **Case 3 applies**

Check regularity: $af(n/b) = 4\cdot(n/2)^3 = 4\cdot n^3/8 = n^3/2 \leqslant cn^3$ for $c = 1/2 < 1$ ✓

∴ $T(n) = \Theta(n^3)$

---

2. Asymptotic Analysis

Asymptotic analysis is a method of describing the **limiting behavior** of an algorithm's running time or space requirement as the input size n approaches infinity. It ignores constant factors and lower-order terms.

Key notations:

- **Big-O $O(f(n))$** – Upper bound (worst case)
- **Big-Omega $\Omega(f(n))$** – Lower bound (best case)
- **Big-Theta $\Theta(f(n))$** – Tight bound (average case)

It allows comparison of algorithms independent of hardware or implementation.

---

3. Factors Influencing Running Time of an Algorithm

1. **Input Size (n)** – Larger input generally means more time
2. **Input Type** – Sorted vs unsorted data affects performance

3. **Algorithm Design** – Choice of technique (greedy, D&C, DP)
4. **Data Structures Used** – Array, linked list, tree, etc.
5. **Hardware Speed** – Processor speed (constant factor)
6. **Programming Language** – Interpreted vs compiled
7. **Compiler Optimizations** – Affects constant factors
8. **Nature of Operations** – Arithmetic, comparisons, memory access

---

4. Greedy Choice Property

The **greedy choice property** states that a globally optimal solution can be reached by making **locally optimal (greedy) choices** at each step, without reconsidering previous choices.

- At each step, the algorithm picks the best available option.
- The choice made is **irrevocable** – never reconsidered.
- It must be proven that local greedy choices lead to a global optimum.

Example: In the fractional knapsack problem, always picking the item with the highest value/weight ratio leads to the optimal solution.

---

5. Master's Theorem – Statement & T(n) = 4T(n/2) + n

Statement: Given $T(n) = aT(n/b) + f(n)$, where $a \geq 1$, $b > 1$:

Case	Condition	Result
1	$f(n) = O(n^{(\log_b(a) - \varepsilon)}), \varepsilon > 0$	$T(n) = \Theta(n^{\log_b(a)})$
2	$f(n) = \Theta(n^{\log_b(a)})$	$T(n) = \Theta(n^{\log_b(a)} \cdot \log n)$
3	$f(n) = \Omega(n^{(\log_b(a) + \varepsilon)}), \varepsilon > 0$	$T(n) = \Theta(f(n))$

For T(n) = 4T(n/2) + n:

- $a = 4, b = 2, f(n) = n$
- $\log_b(a) = \log_2(4) = 2$
- $f(n) = n = O(n^{(2-1)})$ → **Case 1 applies**

∴ $T(n) = \Theta(n^2)$

---

6. Best-Case vs Worst-Case Time Complexity

Feature	Best Case	Worst Case
Definition	Minimum time for any input of size n	Maximum time for any input of size n
Notation	Ω (Big-Omega)	O (Big-O)
Input	Most favorable arrangement	Least favorable arrangement
Example (Linear Search)	Element at index 0 → O(1)	Element at last index → O(n)
Usefulness	Rarely used in practice	Most commonly used for guarantees

---

7. Control Abstraction for Divide and Conquer

```
Algorithm DAndC(P)
{
    if Small(P) then
        return S(P);           // Solve directly if small
    else
    {
        divide P into subproblems P1, P2, ..., Pk;
        Apply DAndC to each subproblem;
        return Combine(DAndC(P1), DAndC(P2), ..., DAndC(Pk));
    }
}
```

Steps:

1. **Divide** – Split problem into smaller subproblems
2. **Conquer** – Recursively solve each subproblem
3. **Combine** – Merge solutions into one final solution

Examples: Merge Sort, Quick Sort, Binary Search, Strassen's Matrix Multiplication

---

8. Algorithm – Definition & Time vs Space Complexity

Definition of Algorithm: An algorithm is a finite, well-defined sequence of instructions to solve a problem or perform a computation, which takes input and produces output in a finite amount of time.

Feature	Time Complexity	Space Complexity
Definition	Amount of time (steps) taken	Amount of memory required
Measured in	Number of operations	Units of memory (bytes/words)
Notation	O, Ω, Θ	Same asymptotic notation
Example	Binary Search: $O(\log n)$	Recursive Fibonacci: $O(n)$ stack space
Goal	Minimize execution time	Minimize memory usage

---

9. Properties of Asymptotic Notations

Big-O (Upper Bound): $f(n) = O(g(n))$ if $\exists\, c > 0, n_0$ such that $f(n) \leqslant c \cdot g(n)\ \forall\, n \geqslant n_0$

Big-Omega (Lower Bound): $f(n) = \Omega(g(n))$ if $\exists\, c > 0, n_0$ such that $f(n) \geqslant c \cdot g(n)\ \forall\, n \geqslant n_0$

Big-Theta (Tight Bound): $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ AND $f(n) = \Omega(g(n))$

Properties:

1. **Reflexivity:** $f(n) = O(f(n))$
2. **Transitivity:** If $f = O(g)$ and $g = O(h)$, then $f = O(h)$
3. **Symmetry:** $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$
4. **Transpose Symmetry:** $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$
5. **Constant Factor:** $c \cdot f(n) = O(f(n))$ for any constant $c > 0$

---

10. Algorithm – Definition & Four Characteristics

Definition: An algorithm is a step-by-step, unambiguous procedure that accepts a set of inputs and produces the desired output within a finite number of steps.

Four Important Characteristics of a Good Algorithm:

1. **Finiteness** – Must terminate after a finite number of steps
2. **Definiteness** – Each step must be clearly and unambiguously defined
3. **Input** – Zero or more well-defined inputs
4. **Output** – At least one well-defined output
5. **Effectiveness** – Each operation must be basic, feasible, and executable

PART B – LONG ANSWER QUESTIONS

1. Algorithm Analysis – Key Characteristics

Algorithm Analysis is the process of determining the computational complexity (time and space) of an algorithm to predict its performance.

Why Analyze Algorithms?

- To compare different algorithms solving the same problem
- To predict behavior on large inputs
- To select the most efficient approach

Types of Analysis:

1. **A Priori (Theoretical):** Analysis before implementation using mathematical models
2. **A Posteriori (Empirical):** Analysis after implementation by measuring actual runtime

Key Characteristics of an Algorithm:

1. **Input:** Takes zero or more inputs
2. **Output:** Produces at least one output
3. **Finiteness:** Terminates in finite steps
4. **Definiteness:** Every step is clearly defined
5. **Effectiveness:** Steps are basic and executable
6. **Correctness:** Produces correct output for all valid inputs
7. **Efficiency:** Uses minimum time and space
8. **Generality:** Applicable to a wide range of inputs

---

2. Fractional Knapsack Problem

Given: Capacity W = 50 kg

Item	Value	Weight	Value/Weight
A	60	10	6.0
B	100	20	5.0
C	120	30	4.0

Step 1: Sort by value/weight ratio (descending): A(6.0) > B(5.0) > C(4.0)

Step 2: Greedy Selection:

- Take **Item A** fully: weight used = 10, value = 60, remaining = 40 kg
- Take **Item B** fully: weight used = 20, value = 100, remaining = 20 kg
- Take **Item C** partially: 20/30 of C → value = 120 × (20/30) = **80**

Total Value = 60 + 100 + 80 = 240

Optimal Selection:

- Item A: 10 kg (full) → Value = 60
- Item B: 20 kg (full) → Value = 100
- Item C: 20 kg (2/3 fraction) → Value = 80
- **Total = 240 units (Maximum)**

---

3. Master's Theorem – Detailed Explanation with Examples

Statement: For a recurrence of the form:

> $T(n) = aT(n/b) + f(n)$, where $a \geq 1$, $b > 1$

Compare f(n) with $n^{(\log_b a)}$:

Case 1: f(n) is polynomially smaller

- Condition: $f(n) = O(n^{(\log_b(a) - \varepsilon)})$ for some $\varepsilon > 0$
- Result: **$T(n) = \Theta(n^{\log_b(a)})$**
- Example: $T(n) = 9T(n/3) + n$ → $\log_3 9 = 2$, $f(n) = n = O(n^{(2-1)})$ → **$T(n) = \Theta(n^2)$**

Case 2: f(n) is equal (same order)

- Condition: $f(n) = \Theta(n^{\log_b(a)})$
- Result: **$T(n) = \Theta(n^{\log_b(a)} \cdot \log n)$**

- Example: $T(n) = 2T(n/2) + n \rightarrow \log_2 2 = 1$, $f(n) = n = \Theta(n^1) \rightarrow$ **$T(n) = \Theta(n \log n)$**

Case 3: f(n) is polynomially larger

- Condition: $f(n) = \Omega(n^{(\log_b(a) + \varepsilon)})$ for some $\varepsilon > 0$, AND $af(n/b) \leqslant cf(n)$
- Result: **$T(n) = \Theta(f(n))$**
- Example: $T(n) = 3T(n/4) + n^2 \rightarrow \log_4 3 \approx 0.79$, $f(n) = n^2 = \Omega(n^{(0.79+\varepsilon)}) \rightarrow$ **$T(n) = \Theta(n^2)$**

---

4. $T(n) = 7T(n/2) + n^2$ – Substitution & Master's Theorem

Method 1: Master's Theorem

- $a = 7$, $b = 2$, $f(n) = n^2$
- $\log_b(a) = \log_2 7 \approx 2.807$
- $f(n) = n^2 = O(n^{(2.807 - 0.807)}) \rightarrow$ **Case 1 applies**
- **$\therefore T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$**

Method 2: Substitution Method

Guess: $T(n) = O(n^{\log_2 7})$

Step 1 – Expand:

```
T(n) = 7T(n/2) + n²
     = 7[7T(n/4) + (n/2)²] + n²
     = 7²T(n/4) + 7(n²/4) + n²
     = 7³T(n/8) + 7²(n²/16) + 7(n²/4) + n²
```

After k levels:

```
T(n) = 7^k · T(n/2^k) + n² · Σ(7/4)^i  [i = 0 to k-1]
```

Step 2 – Base case at $k = \log_2 n$:

```
T(1) = c (constant)
7^(log₂n) = n^(log₂7)
```

Step 3 – Sum the geometric series: Since $7/4 > 1$, the series is dominated by the last term:

```
Σ(7/4)^i = O((7/4)^(log₂n)) = O(n^(log₂(7/4))) = O(n^(log₂7 - 2))
```

So: $T(n) = n^{(\log_2 7)} + n^2 \cdot n^{(\log_2 7 - 2)} = \mathbf{\Theta(n^{\log_2 7})}$

---

5. Kruskal's Algorithm – Minimum Spanning Tree

Algorithm Steps:

1. Sort all edges by weight in ascending order
2. Pick the smallest edge; add it to MST if it doesn't form a cycle
3. Repeat until (V−1) edges are selected (V = number of vertices)
4. Use **Union-Find** to detect cycles

Pseudocode:

```
Sort edges by weight
MST = {}
For each edge (u, v) in sorted order:
    if find(u) ≠ find(v):
        Add (u, v) to MST
        union(u, v)
Return MST
```

Time Complexity: O(E log E) due to sorting

(Apply this algorithm to the graph provided in your question paper by listing edges with weights, sorting them, and greedily adding edges that don't form cycles.)

---

6. Time-Space Trade-off

Definition: The time-space trade-off refers to the concept where an algorithm can be made faster by using more memory, or made to use less memory at the cost of being slower.

Significance in Algorithm Design:

1. **Memory for Speed:** Precomputed lookup tables (e.g., memoization in DP) use more space but drastically reduce computation time.
2. **Speed for Memory:** Algorithms like in-place sorting (e.g., Bubble Sort) use O(1) space but may take $O(n^2)$ time.
3. **Practical Examples:**

Approach	Time	Space	Example
Dynamic Programming	$O(n^2)$	$O(n^2)$	Matrix Chain Multiplication
Recursion without memo	$O(2^n)$	$O(n)$	Fibonacci
Lookup Table	$O(1)$	$O(n)$	Precomputed values

4. **Cache & Hash Maps:** Hash maps $O(1)$ lookup at the cost of $O(n)$ space.
5. **Design Principle:** Choose the trade-off based on the constraints of the system (memory-limited embedded systems vs. performance-critical servers).

---

7. Job Sequencing with Deadlines

Algorithm:

1. Sort jobs by profit in descending order
2. For each job, assign it to the latest available time slot ⩽ deadline
3. If no slot available, skip the job

Given:

- n = 7, Profits: (3, 5, 20, 18, 1, 6, 30), Deadlines: (1, 3, 4, 3, 2, 1, 2)

Step 1 – Create job table:

Job	Profit	Deadline
J7	30	2
J3	20	4
J4	18	3
J6	6	1
J2	5	3
J1	3	1
J5	1	2

Step 2 – Schedule (max deadline = 4, slots: [1,2,3,4]):

Job	Deadline	Slot Assigned
J7	2	Slot 2 ✓
J3	4	Slot 4 ✓
J4	3	Slot 3 ✓
J6	1	Slot 1 ✓
J2	3	No slot (3 taken) ✗
J1	1	No slot (1 taken) ✗
J5	2	No slot (2 taken) ✗

Optimal Sequence: [J6, J7, J4, J3] (Slots 1, 2, 3, 4)

Maximum Profit = 6 + 30 + 18 + 20 = 74

---

8. Substitution vs Recursion Tree vs Master's Theorem

Feature	Substitution Method	Recursion Tree Method	Master's Theorem
Approach	Guess solution, verify by induction	Visualize recursion as a tree	Apply formula directly
Steps	1. Guess T(n) 2. Prove by induction	1. Draw tree 2. Sum all levels	Compare f(n) with n^log_b(a)
Ease	Requires insight to guess	Intuitive, visual	Fastest – direct application
Accuracy	Exact if guess is correct	Approximate (then prove)	Exact for applicable cases
Limitations	Hard to guess; error-prone	Tedious for many levels	Not applicable to all forms
Best for	Verifying a known solution	Understanding structure	Quick complexity determination
Example	T(n) = T(n−1)+1 → O(n)	Merge Sort tree → O(n log n)	T(n) = aT(n/b)+f(n)

---

9. Merge Sort – Recurrence & Worst Case Complexity

Recurrence Relation:

> **$T(n) = 2T(n/2) + \Theta(n)$** $T(1) = 1$

- **$2T(n/2)$:** Two recursive calls on halves
- **$\Theta(n)$:** Merging two halves takes linear time

Derivation using Recursion Tree:

```
Level 0:  n              → cost: n
Level 1:  n/2 + n/2      → cost: n
Level 2:  n/4+n/4+n/4+n/4 → cost: n
...
Level k:  n/2^k          → cost: n
```

- Total levels = $\log_2 n$ (tree height)
- Cost per level = n
- **Total cost = $n \times \log_2 n$**

Using Master's Theorem:

- $a = 2, b = 2, f(n) = n$
- $\log_b(a) = \log_2 2 = 1$
- $f(n) = n = \Theta(n^1)$ → **Case 2 applies**
- **$\therefore T(n) = \Theta(n \log n)$**

Worst-case complexity of Merge Sort = $O(n \log n)$ (Same as best and average case – Merge Sort is always $O(n \log n)$)

---

10. Greedy Job Scheduling – Max Profit

Given:

Job	Deadline	Profit
F1	5	200
F2	3	180
F3	3	190
F4	2	300
F5	1	120

Step 1 – Sort by profit (descending):

Job	Profit	Deadline
F4	300	2
F1	200	5
F3	190	3
F2	180	3
F5	120	1

Step 2 – Schedule (max deadline = 5, slots: 1 to 5):

Job	Deadline	Slot Assigned
F4	2	Slot 2 ✓
F1	5	Slot 5 ✓
F3	3	Slot 3 ✓
F2	3	Slot 1 ✓ (latest free ⩽ 3)
F5	1	No slot (1 taken) ✗

Optimal Job Sequence: [F2, F4, F3, F1] (Slots 1, 2, 3, 5)

Maximum Profit = 180 + 300 + 190 + 200 = 870

End of DAA Answer Bank – 5th Semester BE